

写完了随机数发生器之后，就可以正式测试 `sort` 函数了，程序如下：

```
void test_sort(vector<int>& v) {
    sort(v.begin(), v.end());
    for(int i = 0; i < v.size()-1; i++)
        assert(v[i] <= v[i+1]);
}
```

新内容是上面的 `assert` 宏，其用法是“`assert(表达式)`”，作用是：当表达式为真时无变化，但当表达式为假时强行终止程序，并且给出错误提示。当然，上述程序也可以写成“`if(v[i] > v[i+1]) { printf("Error: v[i]>v[i+1]!\n"); abort(); }`”，但 `assert` 更简洁，而且可以知道是由代码中的哪一行引起的，所以在测试时常常使用它。

提示 5-22：测试时往往使用 `assert`。其用法是“`assert(表达式)`”，当表达式为假时强行终止程序，并给出错误提示。

和刚才一样，给参数 `v` 加上引用符的原因是为了避免 `vector` 复制，但函数执行完毕之后 `v` 会被 `sort` 改变。如果调用者不希望这个 `v` 被改变，就应该去掉“`&`”符号（即参数改成 `vector<int> v`），改回传值的方式。

下面是主程序，请注意 `srand` 函数的调用位置。顺便我们还测试了 `sort` 的时间效率，发现给 10^6 个整数排序几乎不需要时间。

```
int main() {
    vector<int> v;
    fill_random_int(v, 1000000);
    test_sort(v);
    return 0;
}
```

`vector`、`set` 和 `map` 都很快^①，其中 `vector` 的速度接近数组（但仍有差距），而 `set` 和 `map` 的速度也远远超过了“用一个 `vector` 保存所有值，然后逐个元素进行查找”时的速度。`set` 和 `map` 每次插入、查找和删除时间和元素个数的对数呈线性关系，其具体含义将在第 8 章中详细讨论。尽管如此，在一些对时间要求非常高的题目中，STL 有时会成为性能瓶颈，请读者注意。

5.3 应用：大整数类

在介绍 C 语言时，大家已经看到了很多整数溢出的情形。如果运算结果真的很大，就需要用到所谓的高精度算法，即用数组来储存整数，并模拟手算的方法进行四则运算。这些算法并不难实现，但是还应考虑一个易用性问题——如果能像使用 `int` 一样方便地使用大

^① 注意 `vector` 并不是所有操作都快。例如 `vector` 提供了 `push_front` 操作，但由于在 `vector` 首部插入元素会引起所有元素往后移动，实际上 `push_front` 是很慢的。

整数，那该有多好！相信读者已经想到解决方案了，那就是使用 `struct`。

5.3.1 大整数类 BigInteger

结构体 `BigInteger` 可用于储存高精度非负整数。

```
struct BigInteger {
    static const int BASE = 1000000000;
    static const int WIDTH = 8;
    vector<int> s;

    BigInteger(long long num = 0) { *this = num; } //构造函数
    BigInteger operator = (long long num) {      //赋值运算符
        s.clear();
        do {
            s.push_back(num % BASE);
            num /= BASE;
        } while(num > 0);
        return *this;
    }

    BigInteger operator = (const string& str) {   //赋值运算符
        s.clear();
        int x, len = (str.length() - 1) / WIDTH + 1;
        for(int i = 0; i < len; i++) {
            int end = str.length() - i*WIDTH;
            int start = max(0, end - WIDTH);
            sscanf(str.substr(start, end-start).c_str(), "%d", &x);
            s.push_back(x);
        }
        return *this;
    }
};
```

其中，`s` 用来保存大整数的各个数位。例如，若是要表示 1234，则 `s={4,3,2,1}`。用 `vector` 而非数组保存数字的好处显而易见：不用关心这个整数到底有多大，`vector` 会自动根据情况申请和释放内存。

上面的代码中还有赋值运算符，有了它就可以用 `x = 123456789` 或者 `x = "1234567898765432123456789"` 这样的方式来给 `x` 赋值了。

提示 5-23： 可以给结构体重载赋值运算符，使得用起来更方便。

之前已经介绍过 “<<” 运算符，类似的还有 “>>” 运算符，代码一并给出。

```
ostream& operator << (ostream &out, const BigInteger& x) {
    out << x.s.back();
```

```

for(int i = x.s.size()-2; i >= 0; i--) {
    char buf[20];
    sprintf(buf, "%08d", x.s[i]);
    for(int j = 0; j < strlen(buf); j++) out << buf[j];
}
return out;
}

istream& operator >> (istream &in, BigInteger& x) {
    string s;
    if(!(in >> s)) return in;
    x = s;
    return in;
}

```

这样，就可以用 `cin>>x` 和 `cout<<x` 的方式来进行输入输出了。怎么样，很方便吧？不仅如此，`stringstream` 也“自动”支持了 `BigInteger`，这得益于 C++ 中的类继承机制。简单地说^①，由于“>>”和“<<”运算符的参数是一般的 `istream` 和 `ostream` 类，作为“特殊情况”的 `cin/cout` 以及 `stringstream` 类型的流都能用上它。

上述代码中还有两点需要说明。一是 `static const int BASE = 100000000`，其作用是声明一个“属于 `BigInteger`”的常数。注意，这个常数不属于任何 `BigInteger` 类型的结构体变量，而是属于 `BigInteger` 这个“类型”的，因此称为静态成员变量，在声明时需要加 `static` 修饰符。在 `BigInteger` 的成员函数里可以直接使用这个常数（见上面的代码），但在其他地方使用时需要写成 `BigInteger::BASE`。

提示 5-24：可以给结构体声明一些属于该结构体类型的静态成员变量，方法是加上 `static` 修饰符。静态成员变量在结构体外部使用时要写成“结构体名::静态成员变量名”。

5.3.2 四则运算

这部分内容和 C++ 本身关系不大，但是由于高精度类非常常见，这里仍然给出代码（定义在结构体内部）：

```

BigInteger operator + (const BigInteger& b) const {
    BigInteger c;
    c.s.clear();
    for(int i = 0, g = 0; ; i++) {
        if(g == 0 && i >= s.size() && i >= b.s.size()) break;
        int x = g;
        if(i < s.size()) x += s[i];

```

^① 任何一本 C++ 语言教材都会介绍类继承，但它在算法竞赛中很少使用，所以这里略去细节。

```

        if(i < b.s.size()) x += b.s[i];
        c.s.push_back(x % BASE);
        g = x / BASE;
    }
    return c;
}

```

为了让使用更加简单（还记得之前为什么要修改 `sum` 函数吗？），还可以重新定义“+”运算符（定义在结构体内部）：

```

BigInteger operator += (const BigInteger& b) {
    *this = *this + b; return *this;
}

```

减法、乘法和除法的原理类似，这里不再赘述，请读者参考代码仓库。

5.3.3 比较运算符

下面实现“比较”操作（定义在结构体内部）：

```

bool operator < (const BigInteger& b) const {
    if(s.size() != b.s.size()) return s.size() < b.s.size();
    for(int i = s.size()-1; i >= 0; i--)
        if(s[i] != b.s[i]) return s[i] < b.s[i];
    return false; //相等
}

```

一开始就比较两个 `BigInteger` 的位数，如果不相等则直接返回，否则直接从后往前比较（因为低位在 `vector` 的前面）。注意，这样做的前提是两个数都没有前导零，否则，很可能出现“运算结果都没问题，但一比较就出错”的情况。

只需定义“小于”这一个符号，即可用它定义其他所有比较运算符（当然，对于 `BigInteger` 这个例子来说，“=”可以直接定义为 `s == b.s`，不过不具一般性）：

```

bool operator > (const BigInteger& b) const{ return b < *this; }
bool operator <= (const BigInteger& b) const{ return !(b < *this); }
bool operator >= (const BigInteger& b) const{ return !(*this < b); }
bool operator != (const BigInteger& b) const{ return b < *this || *this < b; }
bool operator == (const BigInteger& b) const{ return !(b < *this) && !(*this < b); }

```

可以同时用“<”和“>”把“!=”和“==”定义得更加简单，读者可以自行尝试。

还记得 `sort`、`set` 和 `map` 都依赖于类型的“小于”运算符吗？现在它们是不是已经自动支持 `BigInteger` 了？赶紧试试吧！